Backend and AI engineering brief

How to replace the browser-only prototype with a real control plane: Convex as the backend, harness agents behind a spend-capped inference fence, and a strict per-site tenancy model. Built in ten handoffs without changing what the UI shows.

CONTROL PLANE	MODELS	AUTONOMY	PHASES
Convex + Workflow	Anthropic direct	L3 · exception queue	B0 → B9

01 · STARTING POINT

What exists and what replaces it

The frontend is complete. Its data lives in `src/lib/store.tsx` with timers standing in for agents. The backend replaces the store behind the same hooks, so `src/features/**` doesn't change.

PROTOTYPE TODAY	BACKEND REPLACEMENT
localStorage state, cross-tab sync	Convex tables (41 drafted) and live queries
"Pick a person" sign-in, per-tab user	Clerk with Organizations: one org = one workspace
Timers simulating agent jobs	Workflow and Workpool components, crons
Next.js API routes for connections, templates, visual agent (localhost only)	Convex Node actions; the API routes are deleted
Template bundles and keys in `.data/` on disk	R2 storage; per-site `secrets` table with AES-256-GCM and key versions
Simulated merge requests	GitHub App (installation tokens, path allowlist, webhooks); GitLab app
Seeded traffic and KAS	Daily Google Search Console import

Reference docs in the repo: `docs/engineering-blueprint.md` (stack and constraints) · `docs/backend/README.md` (design) · `schema.md` and `convex-schema.draft.ts` (tables, typechecks) · `api-map.md` (every store action → Convex function) · `handoffs/00-09`.

02 · STACK

Framework defaults, with justified departures

LAYER	CHOICE	NOTE
Shell and deploy	Next.js 15 App Router on Vercel; Convex prod	Framework default
Auth	Clerk with Organizations	Site roles live in Convex; Clerk has no per-resource roles
Control plane	Convex, own deployment at `apps/cms/convex`	Decision D1
Durable jobs	Workflow, Workpool, crons	Replaces the PRD's Postgres + queue (C2)
Storage	Convex R2 component; `sharp` in Node actions	Bundles up to 50 MB / 2,000 files (C3)
Models	Anthropic API direct; OpenRouter as fallback adapter	Harness pins Anthropic IDs and uses structured outputs (C1, D2)
Money and inference	`@studio/effect-critical`, new `cappedInference`	Framework rule: Effect fences money and inference
Billing	Polar subscriptions + Autumn credits; site ledger is truth	Billing unit is decision D3
Runtime agents	`@studio/ai-runtime` budgets only, no sandbox	Agents never execute code (C6)
MCP	HTTP MCP only (Streamable HTTP, SSE fallback)	No stdio for customer servers (C5)
Web tools	Parallel → Firecrawl → Browserbase	Fact sourcing and research, cheapest first
Observability	Langfuse, Sentry, PostHog, audit table	One Langfuse trace per run
Review and security	Rams, Greptile + greploop, Aikido	Aikido mandatory before pilot: repo write + secrets

Layering

Application	`apps/cms/convex/*` : access checks (`requireSiteRole`), post state machine (`posts.transition`), gate and autopost decisions (reused from `logic.ts`), audit, notifications.
Capabilities	`@studio/seo-harness` (roster, routing, skills) · `@studio/web-tools` · `@studio/storage` R2 port · `src/render` renderer · pure parts of the visual agent.
Effect fence	`cappedInference` for all text and image spend · spend ledger debit, credit, reconcile · message delivery (v1.1).
Platform	Convex (+ Workflow, Workpool, R2, Rate Limiter, Autumn, Polar) · Clerk · Vercel · Anthropic · image providers · GitHub/GitLab · Google · Langfuse · Sentry · PostHog.

Call shape everywhere: **action → Effect program → mutation**. Actions orchestrate and return typed results; one internal mutation persists each step. The browser only ever calls Convex functions with the Clerk token.

Tenancy and isolation

Workspace = Clerk organisation. Site roles are Convex `memberships`. Effective role: workspace owner or admin → admin on every site; otherwise the membership role; otherwise no access. Every public site function is registered through a wrapper that calls one helper first:

```
requireSiteRole(ctx, siteId, permission)
  1  identity from the Clerk token            → "Not authenticated"
  2  user row, not suspended
  3  site row                                 → "Not found" (same as no access)
  4  site.workspaceId = token's active org
  5  role = effectiveRole(workspace role, site membership)
  6  PERMISSIONS[permission] includes role    → FORBIDDEN + the UI's lock reason
```

#	ISOLATION RULE
I1	Every public site function takes `siteId` first and checks access before any read
I2	Child records are read through their parent, after checking the parent belongs to the site
I3	Tenant tables are only queried by a `by_site` index; no whole-table reads
I4	Agent context is built from one site only; a step refuses input from another site
I5	R2 keys are prefixed `sites/<siteId>/` ; URLs minted only after the access check
I6	Secrets decrypt only inside internal actions and never reach prompts, logs, audit or Langfuse
I7	Webhooks resolve the tenant from the provider payload, never from the URL
I8	"Doesn't exist" and "not yours" return the same error

04 · POST LIFECYCLE

A chain of short workflows, not one long one

States and transitions stay exactly as in `logic.ts`. A human pause (review, merge) is a row state, not a sleeping workflow. Only `posts.transition(postId, from, to)` moves a post, and it checks `from`, so a late step can't undo what a person already did.

approveIdea
Access check, idea → briefed, audit

→

postPipeline
Draft, optimise, fan out visual agent per image slot (Workpool)

→

autopostGate
evaluateGate → fixes → re-evaluate → ship or hold

→

shipPost
Render → branch `content/<slug>` → commit → open PR

→

Webhooks
CI green → auto-merge → live → KAS after 24 h

Holds

Autopost off, no repo, no confirmed template or it can't render the post, over cap, provider failure, an image slot for a person, an unfixable hard check. Each writes an "Autopost held" ledger item and notifies specialists.

Scheduling

One cron ticks every 5 minutes over per-site schedule rows (in the site's timezone, pausable). The Haiku orchestrator is replaced by deterministic code (D6). "Run now" uses the same workflow as the cron.

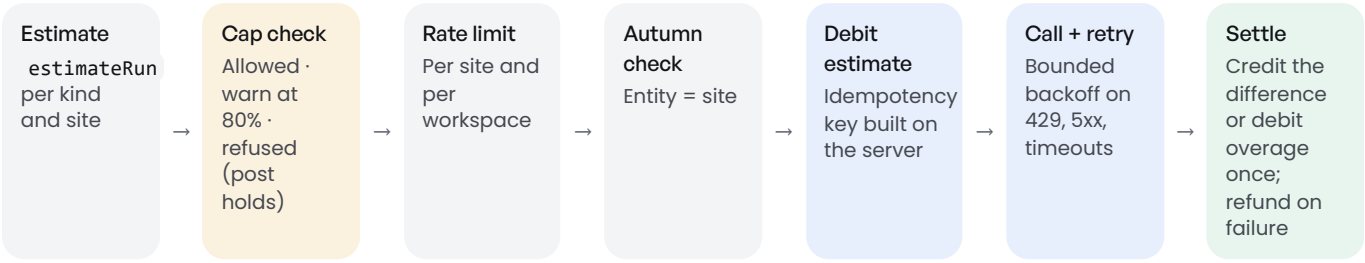
05 · AI ENGINEERING

Agent execution model

CONCEPT	IMPLEMENTATION
Agent = workflow step	A Node action `agents.<id>.run({ siteId, runId, inputs })`: builds single-site context, calls the model through `cappedInference`, validates the structured output and returns it. The next mutation persists it.
CMS agent	Not an LLM loop. It is the workflows, `posts.transition` and `gitNode.*` (the only repo writer). It calls a model only for autopost fixes, at the workhorse tier.
Prompts	Each `skills/<id>/SKILL.md` compiled to `skills.generated.ts` at build time (Convex can't read package files at runtime). System prompt = skill + site brand pack + guardrails. Skill hash stored on the run.
Structured outputs	A zod schema per agent artefact (writer: blocks, facts, image sheet, word count; optimizer: gate checks; swarm: keyword rows), via Anthropic structured outputs, validated again on the server.
Tier routing	Per-site `modelRouting` (mechanical · writing · strategy · pillar pages) → `fast` Haiku 4.5 · `workhorse` Sonnet 5 · `strongest` Opus 5.5. Verify IDs before deploy.
Escalation	Static from the harness config (writer → strongest for pillar and money pages; report → strongest on pivot). Dynamic (D7): invalid output → retry once → one tier up if the cap allows → else fail and hold.
Budgets	`assertWithinBudget` before every turn of a multi-turn step; `maxTurns` set explicitly per agent.
Tools	Web research cheapest-first; image providers through HTTP MCP or APIs. No code execution, so no sandbox. Agent SVG goes through `sanitizeSvg()`.
Guardrails	Fact markers for unsourced claims, regulated disclaimers, no invented credentials. Images: no real people, no living-artist styles, no watermarks, licence check and provenance, estimate before running.

06 · THE INFERENCE FENCE

Every paid call goes through one program



- Two monthly budgets per site: text and image, in USD, set by an admin. Over cap refuses with a message; nothing is billed silently.
- Ledger truth is `spendEntries` in integer micro-USD. Autumn is the commercial meter, reconciled daily.
- Customer-supplied provider keys are still estimated and capped, but debited as zero provider cost plus any CMS fee (D15).

Observability

SIGNAL	CONTENT
Langfuse	One trace per run, spans per model or tool call; tagged site, workspace, agent, tier, skill hash. No secrets, no personal data.
Sentry	Server and client errors tagged with site, run and workflow
PostHog	`idea_approved`, `post_shipped`, `post_held` (reason), `mr_merged` (autopost), `run_refused_over_cap`, `slot_filled` (route)
Audit	Append-only, customer-facing: "who approved this claim" is one query
Runs	Cost, tokens, model and status per run, for per-client cost reporting

Failure handling

FAILURE	BEHAVIOUR
Model call fails	Bounded retry inside the fence → refund → run failed → post holds "Provider failure"
Invalid structured output	Retry once, escalate one tier, else fail as above
Image provider fails	Retry once, try the next route, else hold the slot
Template can't render	Hold "Template can't render", ledger item for the developer
Base branch moved	Rebase re-renders from the current template; a conflict notifies the developer; never force onto base
CI red	MR stays open with CI detail on the timeline; autopost doesn't merge
Webhook replayed or out of order	Deduplicated by delivery ID; stale head SHAs ignored
Search Console token expired	Shows "reconnect"; KAS shows a data gap, never zeros

07 · BUILD ORDER

Ten handoffs

Each handoff in `docs/backend/handoffs/` is self-contained for a builder agent: goal, inputs to read, files to create, acceptance tests and exit criteria. Run them through the compound-engineering loop.

PHASE	HANDOFF	DEPENDS ON	EXIT
B0	Setup: accounts, env tiers, keys	—	Three tiers provisioned; `.env.example` and env matrix merged
B1	Foundations and tenancy	B0	Isolation sweep green; workspace, site, team and settings on Convex
B2	Post lifecycle workflow	B1	Every state edge tested with stub agents
B3	Agents and models	B2	Real agents behind the fence; staging eval recorded
B4	Visual agent and assets	B1–B3	Four image routes from Convex; Next.js API routes removed
B5	Templates and renderer	B1, B4	Fidelity holds; HTML, MDX and JSON golden files
B6	Git and merge	B2, B4, B5	Approve → PR → green CI → live on staging
B7	Search Console and KAS	B1, B2, B4	Real trajectory; KAS within 24 h of merge
B8	Spend and billing	B1	Caps hold under concurrency; daily reconcile
B9	Hardening and pilot	B1–B8	Aikido clear; pilot exit criteria met

Parallel after B2: B3, B7 and B8 together. B4 needs B3's gateway. B5 parsing can start after B1. B6 follows B5.

Environment tiers

A · Boot

Convex deployment and URL; Clerk issuer, publishable and secret keys, webhook secret; frontend URL. Clerk JWT template `convex`, Organizations on.

B · Money

Polar access token, organisation ID, webhook secret, server; Autumn secret key.

C · AI jobs

`ANTHROPIC_API_KEY`; Langfuse keys and URL; `CMS_SECRETS_KEY_V1` and `CMS_SECRETS_KEY_CURRENT` for key rotation.

D/E · Observability and integrations

PostHog, Sentry; Parallel, Firecrawl, Browserbase; GitHub App; Google OAuth; R2 credentials.

Every server key lives in Convex env; Next.js only holds public keys and Clerk. Per-client keys go in the encrypted `secrets` table.

Open decisions

#	DECISION	RECOMMENDATION	BLOCKS
D1	Own Convex deployment or share apps/web's	Own	B1
D2	Anthropic direct or via OpenRouter	Direct; OpenRouter as fallback	B3
D3	Bill per site or per workspace	Workspace plan with seats + per-site subscription item	B8
D4	Autumn customer mapping	Customer = workspace, entity = site	B8
D5	Who merges on green CI	The CMS, from the webhook (re-checks the gate)	B6
D7	Dynamic escalation	Retry, then one tier up within cap	B3
D8	Path scope enforcement	CMS allowlist plus branch protection in the repo	B6
D14	Image licensing policy	Product call, per provider	Paid pilot
D15	Cost model and customer-key billing	Product call	B8

The full list (D1–D17) is in `docs/backend/README.md` §15. Still missing for the venture commit: a signed eval-first spec (L3 autonomy and cost-per-post budget) and the problem scorecard.